DESARROLLO DE APLICACIONES II

Pendientes para cerrar la respuesta a M2

Equipo 9 — Core · Desarrollo de Aplicaciones II

27 de octubre de 2026

Contenido

1 · ¿Ciudadanos tiene que recibir `updateTicketStatus`? 3
2 · Cambiar el `ownerModule` a `atencion-ciudadana` 3
3 · Validar `sourceModule` contra la identidad del productor en AMQP 3
4 · Retención de `eventId` y versiones de schema 4
5 · `PROGRESS` que llega antes de `STARTED` 4
6 · Validación de `format` y restricciones del sobre de M2 5
7 · Permisos sobre schemas y propietarios · **crítico** 6
Lo operativo 7
Resumen de lo que proponemos 7

Dos cosas antes de los siete puntos.

No pude verificar el estado del Core desplegado. Las tres suscripciones pausadas y los tres schemas v1.78 quedaron en el servidor, y no tengo credenciales de ese ambiente. Todo lo que dice este documento sobre *datos* sale de la base de desarrollo; todo lo que dice sobre *comportamiento* sale del código y está verificado ejecutándolo.

El punto 7 es el más urgente y es peor de lo que creíamos. No es sólo que cualquiera pueda reescribir un schema ajeno: **cualquier módulo autenticado puede apropiarse del tipo de evento de otro.** Lo probamos. Está abajo.

---

1 · ¿Ciudadanos tiene que recibir `updateTicketStatus`?

Estado hoy (base de desarrollo):

```
updateTicketStatus
  lo publican : ciudadanos, desarrollo-social, obras, rentas, transito
  lo escucha  : atencion-ciudadana
```

Cinco módulos lo publican y uno solo lo consume. Que el paquete de M2 lo mencione para Ciudadanos no alcanza para darlo de alta: **la suscripción la crea cada equipo** con su token, y es el acto por el cual el Core declara su cola y su binding.

Lo que proponemos: no lo damos de alta nosotros. Que Ciudadanos confirme si lo necesita y lo cree con `POST /api/v1/subscriptions`. Si lo creamos nosotros, Ciudadanos empieza a recibir tráfico que no pidió y que quizá su consumidor no sabe procesar — y cada mensaje que rechace entra en la cadena de reintentos.

Vale una pregunta de diseño: Ciudadanos **publica** `updateTicketStatus` y además lo escucharía. Un módulo suscripto a un tipo que él mismo emite va a recibir sus propios eventos. Si eso no es lo que quieren, conviene revisarlo antes de abrir la suscripción.

---

2 · Cambiar el `ownerModule` a `atencion-ciudadana`

A favor del cambio: `ownerModule` debería ser quien define el contrato del tipo. El único consumidor es Atención Ciudadana, y es el que sufre si el `data` cambia de forma. Que figure `ciudadanos` como dueño siendo uno de cinco productores es arbitrario.

En contra, y es lo que nos frena: el dueño de un evento normalmente es quien lo **publica**, no quien lo consume. Acá hay cinco productores, así que el tipo no tiene un dueño natural: es un evento de notificación compartido.

Lo que proponemos: dejarlo en manos del que define el contrato. Si el schema de `updateTicketStatus` lo escribe Atención Ciudadana —que parece el caso, por el v1.78— entonces que figure como `ownerModule`, y los cinco productores se alinean a ese contrato. Pero **que lo confirmen los cinco**, no sólo M2: cambiar el dueño cambia quién puede romperles la validación.

Y ojo con el orden: **esto no se puede hacer hasta cerrar el punto 7.** Hoy ese cambio lo puede hacer cualquiera, y deshacerlo también.

---

3 · Validar `sourceModule` contra la identidad del productor en AMQP

Hoy no se valida. Y es importante ser exactos: el 403 que documentamos es del camino HTTP.

Camino	Autenticación	¿Verifica `sourceModule`?
`POST /api/v1/events`	Bearer JWT	**Sí** — otro módulo en el sobre → 403
Publicar en `muni.inbox`	ninguna	**No** — se procesa tal cual viene

El worker que consume `core.inbox` valida la *forma* del sobre y nada más: no recibe token y no compara el `sourceModule` con nada.

Cómo se cierra. El despliegue ya tiene lo que hace falta: un usuario de RabbitMQ por módulo. Falta el último paso, que es activar el mecanismo que el propio broker trae:

1. En el publisher, marcar el mensaje con `user_id` (aio-pika: `user_id=` en el `Message`). Precisión (27/10, verificado contra el broker): RabbitMQ valida el `user_id` **cuando viene** —rechaza si no coincide con el usuario de la conexión— pero **no rechaza su ausencia** por defecto. Así que para que esto sea una garantía hay que **exigirlo**, no solo permitirlo.
2. En el worker, comparar ese `user_id` contra `envelope.sourceModule` y rechazar si difieren. Este paso es nuestro y todavía no está implementado.

Los dos pasos juntos dan la garantía; hoy no está ninguno activo.

Cuándo. Es un cambio chico de nuestro lado, pero **rompe a todo módulo que hoy publica sin `user_id`**. Hay que avisarlo con fecha y darles una ventana. Propuesta: lo anunciamos ahora, lo activamos en modo advertencia —se registra la discrepancia pero no se rechaza— y recién después lo hacemos obligatorio.

---

4 · Retención de `eventId` y versiones de schema

Retención

Hoy no hay purga: `event_log` crece sin límite. No es un olvido cómodo, es una decisión con consecuencia: **la idempotencia depende de esa tabla.** El `UNIQUE(event_id)` es lo que hace que reenviar un evento no duplique el efecto.

Entonces la ventana de deduplicación es, hoy, *para siempre*. El día que purguemos, pasa a ser el período de retención: un evento reenviado después de ese plazo se procesaría como nuevo.

Lo que proponemos: retención de **90 días para el sobre completo** y **indefinida para el `eventId`**. Son dos cosas distintas y conviene separarlas: el `data` es lo que ocupa y lo que puede traer datos personales; el `eventId` es 32 bytes y es lo que sostiene la garantía. Un evento reenviado un año después sigue detectándose como duplicado, pero su contenido ya no está.

Hasta que eso exista, lo honesto es decir: **no hay política de retención y la tabla crece**.

Versiones de schema

El sobre ya trae `eventVersion`, pero hoy **el schema se guarda por tipo, no por versión**: cargar uno nuevo pisa el anterior. Por eso lo del v1.78 es un cambio destructivo aunque no lo parezca.

Lo que proponemos: que un cambio incompatible sea un **tipo nuevo** (`updateTicketStatusV2`), no un schema nuevo sobre el mismo nombre. Es más tosco que versionar el schema, pero deja que productores y consumidores migren a su ritmo: durante la transición los dos tipos conviven y cada uno se suscribe al que sabe procesar. Con un solo nombre, el día que se carga el schema nuevo **todos los productores viejos empiezan a ir a la DLQ a la vez**.

---

5 · `PROGRESS` que llega antes de `STARTED`

No hay una operación de posposición explícita, pero **sí hay el mecanismo: es el nack.**

Cuando un consumidor hace `nack`, el Core no descarta el evento. Lo manda al escalón de backoff que corresponda y el broker se lo vuelve a entregar cuando vence el TTL. O sea: **nack = «todavía no puedo con esto, traémelo más tarde».**

> **Corregido el 2026-10-27.** La version original de esta respuesta decia `15 s / 1 m / 5 m / 15 m` y una ventana de ~21 minutos. Estaba mal: ese era el valor de un `.env` local que usamos para la demo, no el desplegado. Los numeros de abajo son los correctos, y son **un orden de magnitud menores**.
>
> Los escalones reales se consultan en `GET /api/v1/topology`, que devuelve cada cola de espera con su `x-message-ttl`. Si no coincide con esto, gana el endpoint.

```
nack #1 ──► vuelve en   5 s
nack #2 ──► vuelve en  30 s
nack #3 ──► vuelve en   2 min
nack #4 ──► dead letter, visible en GET /api/v1/dlq
```

`maxAttempts` cuenta **entregas, no reintentos**: con el default de 4 son la entrega original mas 3 reintentos, y el cuarto nack agota. Por eso el cuarto escalon de la cadena (10 min) no llega a usarse; haria falta `maxAttempts = 5`.

Lo que recomendamos a M2 para este caso: si llega `PROGRESS` y no está el `STARTED`, hacer `nack`. Tienen **tres reintentos repartidos en 2 min 35 s** para que llegue el que falta. Alcanza para el desfasaje normal de un par de mensajes en vuelo, pero **no** para un consumidor que estuvo caido un rato. Si necesitan una ventana mas larga, se sube el `maxAttempts` de esa suscripcion: con 5 pasa a 12 min 35 s.

Dos límites que conviene que sepan:

- **Agotados los cuatro, va a la DLQ.** No se pierde y se puede reprocesar a mano, pero deja de ser automático.
- **El nack no distingue el motivo.** Para el Core, «no puedo todavía» y «esto está mal» son lo mismo. Si quieren distinguirlos —reintentar el primero y mandar el segundo directo a la DLQ sin gastar intentos — decímelo y lo vemos: se puede resolver con un header, pero es un cambio de contrato que afecta a los nueve.

---

6 · Validación de `format` y restricciones del sobre de M2

`format` no se valida. Usamos `Draft202012Validator` sin `FormatChecker`, y en JSON Schema `format` es anotación por defecto. Lo pasamos por el validador real:

```
schema: {"cuando": {"type":"string","format":"date-time"},
         "mail":   {"type":"string","format":"email"}}

{"cuando": "esto no es una fecha", "mail": "ni esto un mail"}  -> ACEPTADO
```

O sea: si el schema v1.78 confía en `format` para validar fechas o mails, **esa validación no está ocurriendo**.

Se activa con una línea (`format_checker=Draft202012Validator.FORMAT_CHECKER`), pero tiene una consecuencia que hay que decidir con los nueve: eventos que hoy pasan empezarían a ir a la DLQ. Si lo activamos, que sea anunciado y en una fecha acordada, no de un día para el otro.

Lo que proponemos: activarlo, porque un `format` que no valida es peor que no tenerlo —da una seguridad falsa—, pero con aviso previo. Y mientras tanto, comunicarlo: hoy es la limitación más fácil de pasar por alto.

Sobre las restricciones particulares del sobre de M2: pasanos cuáles son y las revisamos una por una. Lo que ya sabemos que el sobre **sí** hace cumplir: rechaza campos no declarados (`extra="forbid"`), exige `occurredAt` con offset de zona, y valida que los UUID sean interpretables — aunque **no** exige RFC estricto: acepta cualquier versión y variante.

Precisión (27/10) sobre dónde termina un evento inválido —corrige lo que dijimos antes, que mezclaba dos casos:

- **Sobre inválido** (campo no declarado, falta un obligatorio, `occurredAt` sin zona): se rechaza con `422` **en el borde HTTP, antes de entrar al hub**, y **no** queda en la DLQ. El productor se entera en la respuesta; no hay registro que reprocesar.
- **Sobre válido pero** `data` **que no cumple el schema** (incluido `format` si se activa): ahí **sí** entra al hub, se registra y queda en la **DLQ** con el detalle del campo, reprocesable.

Por AMQP no hay `422` : un sobre inválido se descarta y queda en la DLQ como mensaje ilegible; conviene publicar por HTTP mientras se prueba, para ver el error en el acto.

---

7 · Permisos sobre schemas y propietarios · crítico

Esto hay que cerrarlo antes que lo demás, porque varios de los puntos anteriores dependen de que el registro sea confiable.

No hay ningún control de propiedad en el registro de tipos. Lo verificamos ejecutándolo, con el token de `rentas` :

```
owner ANTES                    : ciudadanos
POST /api/v1/event-types       : HTTP 201
owner DESPUES                  : rentas
-> rentas se apropio de un tipo de otro modulo
```

Y el mismo agujero en el schema:

```
'workOrderScheduled' pertenece a: obras
PUT /event-types/workOrderScheduled/schema (token de rentas) -> HTTP 200
-> rentas reescribio el contrato de un evento de obras
```

El impacto está probado, no supuesto. Después de alterar el schema, el dueño publica su propio evento legítimo y recibe `422 SCHEMA_VIOLATION` ; el evento cae en su DLQ. Es una interrupción de servicio contra otro equipo, desde una cuenta común, y la víctima no tiene cómo enterarse salvo mirando su DLQ.

En el código: `RegistryService.set_schema()` no recibe el módulo que llama, y `declare_event_type` toma el `ownerModule` del payload (`owner_module=payload.owner_module or caller.module`). Las suscripciones y las publicaciones **sí** chequean; el registro de tipos es la excepción.

El arreglo: pasar el módulo que llama a las dos operaciones y exigir que sea el dueño **o** el admin. Son pocas líneas y no depende del deploy.

Lo operativo

Los `ticketCreated` que quedaron en las colas pausadas

Pausar una suscripción **no vacía la cola**: los mensajes que ya estaban siguen ahí y se entregan cuando se reactive. Hay tres caminos y conviene elegir uno a conciencia:

Opción	Qué pasa	Cuándo conviene
Dejarlos	Al reactivar, el consumidor los recibe todos de golpe	Si los eventos siguen siendo válidos
Purgar la cola	Se descartan; queda la evidencia en `event_log`	Si son de prueba o ya se procesaron por otra vía
Consumir y descartar	El módulo los saca y los ignora	Si quieren control fino

Lo que proponemos: antes de decidir, mirar **cuántos son y de cuándo**. Si son eventos de prueba de la integración, purgar. Si hay alguno real, no. Podemos pasarles el conteo por cola en cuanto tengamos acceso al desplegado.

Y una advertencia: si se reactivan las tres suscripciones a la vez con cola acumulada, los tres consumidores reciben una ráfaga. Mejor de a uno.

El horario de prueba

Lo definimos nosotros apenas sepamos el de ustedes. **Una sola cosa a tener en cuenta:** hay un solo ambiente, el vhost `core`. No hay separación test/prod, así que **los eventos de prueba les van a llegar a todos los módulos suscriptos**. Conviene avisar en el grupo general antes de empezar, y no al revés.

Resumen de lo que proponemos

#	Propuesta	Depende de
7	**Cerrar los permisos del registro de tipos.** Dueño o admin.	nosotros, pocas líneas
3	Validar `user_id` del mensaje contra `sourceModule`, con ventana de aviso	nosotros + los 9 equipos
2	Cambiar el `ownerModule`, **después** del punto 7	confirmación de los 5 productores
1	Que Ciudadanos cree su suscripción si la necesita	Ciudadanos
6	Activar validación de `format`, con fecha anunciada	acuerdo de los 9
4	Retención 90 días del sobre, indefinida del `eventId`; versión = tipo nuevo	acuerdo del equipo
5	Usar `nack` como posposición: 3 reintentos en 2 min 35 s	nada, ya funciona